EMERGING TECHNOLOGIES | TEAM SPEAKER NOTES

Presentation speaking order

Lâm Xê Khoa

English narration for assigned slides, in the classroom display order.

PRESENTER	ROLE	SLIDES	COVERAGE
Trần Việt Hải	Opening	**1–4**	Introduction, problem, project goals
Lâm Xê Khoa	Main presenter 1	**5–15**	LLMs, RAG, architecture, source PDFs, metadata
Mai Tuấn Dũng	Main presenter 2	**16–26**	Chunking, embeddings, FAISS, pipelines, LCEL, prompting
Sơn Nguyên	Main presenter 3	**27–37**	Ollama, app.py, logging, evaluation, risks, conclusion
Châu Ngọc Đức	Demo lead	**38**	Observed LangChain question; launch/operate demo
Trần Việt Hải	Demo + closing	**39–40**	Out-of-PDF question; instructor questions; closing

Display order

Slides 1–30 → original slides 33–39 → original slides 31–32 → final Questions slide 40. The presentation-order PowerPoint already reflects this sequence.

Duc operates the Windows 11 terminal and introduces the real LangChain screenshot. Hai demonstrates an out-of-PDF question, enters the instructor's questions, and closes the presentation. Both can assist with the live Q&A.

Demonstration safeguard

Use only observed outputs and verify answers against the sample PDF. A retrieved source list alone does not establish that the generated answer is supported. The old slides 31–32 become slides 38–39 in the speaking-order edition.

What Is a Large Language Model?

WHAT TO SAY

A large language model processes a prompt and predicts tokens to form a response. Models are useful for explanation, summarization, and question answering. However, a model does not automatically have access to a private PDF on our computer. Even a fluent answer may be incorrect. Our system therefore uses retrieval to supply relevant document text before generation.

KEY TAKEAWAY

An LLM generates text; it is not a verified document database.

Why Plain LLM Answers Are Not Enough

WHAT TO SAY

A model can generate sentences that sound credible even when the correct evidence is missing. It may not know a new PDF or specific internal document, and it may use general learned knowledge instead. RAG can reduce this problem by supplying retrieved passages. But it does not automatically prevent hallucinations. This distinction becomes especially important when we inspect our own terminal screenshots later in the presentation.

KEY TAKEAWAY

Good wording is not proof of a supported answer.

What Is RAG?

WHAT TO SAY

Retrieval-Augmented Generation means combining retrieval from an external source with language generation. When the user asks a question, the application searches indexed documents for relevant passages. It then inserts these passages into a prompt and asks a model to respond. This allows the model to work with a PDF without training on that PDF. The approach remains dependent on both retrieval quality and answer faithfulness.

KEY TAKEAWAY

RAG retrieves evidence at question time.

Indexing and Question Answering

WHAT TO SAY

Our application has two phases. In the indexing phase, it reads PDFs, splits text, creates embeddings, and builds a FAISS index. It saves the index to disk. At question time, the application uses the saved index to find relevant chunks and sends them to Qwen2.5 through LangChain. The saved index can be reused when the source PDFs and selected settings have not changed.

KEY TAKEAWAY

Index once when needed; retrieve for each question.

Technology Stack in the New Demo

WHAT TO SAY

Our real code uses Python, LangChain integrations, Hugging Face embeddings, FAISS, and Ollama. The language model is Qwen2.5 with the 3-billion-parameter tag, requested as qwen2.5:3b. The embedding model is paraphrase-multilingual-MiniLM-L12-v2. Crucially, generation now goes through ChatOllama and LangChain Expression Language, unlike the older prototype, which called the Ollama package directly.

KEY TAKEAWAY

This is the stack in the uploaded source code, not the old prototype.

Source Code Structure

WHAT TO SAY

The code is organized by responsibility. app.py provides the console interface. config.py stores the active model and retrieval settings. Under src, pdf_reader.py loads text, text_chunker.py splits it, vector_store.py manages embeddings and FAISS, qa.py retrieves and generates answers, and pipeline.py decides whether to rebuild the index. The project also has BAT launchers, a notebook, supporting documents, and contract tests.

KEY TAKEAWAY

The source separates user interaction, indexing, retrieval, and generation.

Windows 11 Setup and Launch

WHAT TO SAY

The uploaded ZIP contains Windows batch files to simplify setup and execution. The first batch file creates a Python virtual environment, installs requirements, and downloads the Ollama model. The second launches the question-answering application. There are also helpers for opening the notebook and checking the environment. The source recommends Python 3.11 and a running local Ollama service. We should always test these steps on the presentation computer before a live demonstration.

KEY TAKEAWAY

The project is designed for straightforward local Windows use.

Configuration Values

WHAT TO SAY

The active values are defined in config.py. Each chunk targets 900 characters with 150 characters of overlap. The retriever requests up to four passages. The local generation model is qwen2.5:3b. The embedding model is the multilingual MiniLM L12 model, selected for multilingual content such as Vietnamese. These are the values in the uploaded configuration file; they should replace all figures inherited from our earlier presentation.

KEY TAKEAWAY

The true demo settings are 900 / 150 / 4, not 1000 / 200 / 4.

Our Sample PDF

WHAT TO SAY

The ZIP includes a three-page PDF about a fictional book club. The first page describes the club, the second describes borrowing rules, and the third lists activities and contact-related information. We verified that all three pages contain extractable text. The file is intentionally a practice dataset, not a real organization. The loader can also process additional PDFs placed in data/pdfs, provided the pages contain selectable text.

KEY TAKEAWAY

The demonstration data is a three-page fictional PDF.

Reading PDFs with PyPDFLoader

WHAT TO SAY

The pdf_reader module finds PDF files and loads them with PyPDFLoader. It processes each page as a LangChain Document and keeps pages that contain nonblank text. It replaces the source metadata with the PDF file name. Page numbering in the metadata begins at zero, so the application later adds one for display. If the PDF is scanned without a text layer, this loader cannot extract the needed content automatically.

KEY TAKEAWAY

Only extractable text becomes searchable.

SLIDE 15 / 40 • IMPLEMENTATION • SPEAKER: Lâm Xê Khoa

Why Metadata Matters

WHAT TO SAY

A retrieved passage needs more than the passage text. It should also carry information about the source PDF, page, and chunk. Our chunker assigns a chunk_id to the split documents, while pdf_reader.py keeps the page metadata. The question-answering module returns this metadata separately from the generated answer. This makes it possible for a user to open the document and check whether a retrieved passage actually supports a claim.

KEY TAKEAWAY

Source metadata enables checking; it does not prove the answer automatically.

SPEAKER HANDOFF / LIVE DEMO LINE

Next, Dung will walk us through the indexing and question-answering implementation.